

LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection

Jiaying Qi, Shaohan Huang, Zhongzhi Luan, Carol Fung, Hailong Yang, Depei Qian

Abstract—The increasing volume of log data produced by software-intensive systems makes it impractical to analyze them manually. Many deep learning-based methods have been proposed for log-based anomaly detection. These methods face several challenges such as high-dimensional and noisy log data, class imbalance, generalization, and model interpretability. Recently, ChatGPT has shown promising results in various domains. However, there is still a lack of study on the application of ChatGPT for log-based anomaly detection. In this work, we proposed LogGPT, a log-based anomaly detection framework based on ChatGPT. By leveraging the ChatGPT’s language interpretation capabilities, LogGPT aims to explore the transferability of knowledge from large-scale corpora to log-based anomaly detection. We conduct experiments to evaluate the performance of LogGPT and compare it with three deep learning-based methods on BGL and Spirit datasets. LogGPT shows promising results and has good interpretability. This study provides preliminary insights into prompt-based models, such as ChatGPT, for the log-based anomaly detection task.

Index Terms—anomaly detection, deep learning, ChatGPT, system log

I. INTRODUCTION

Log-based anomaly detection is an important technique to monitor system activities and identify suspicious behaviors. Logs contain the records of various operations, events, and status information, which are critical for troubleshooting and security analysis. However, manually analyzing large volumes of logs is impractical. In recent years, many automated log-based anomaly detection methods have been proposed, including rule-based methods [1], machine learning (ML)-based methods [2], and deep learning (DL)-based methods. Among these methods, deep learning-based methods [3], [4] have shown supreme performance. They are capable of learning complex patterns and representations from the logs, which allows them to effectively identify anomalies that may not be detected by traditional rule-based or ML-based methods [5].

Deep learning-based methods leverage the power of neural networks and advanced techniques such as recurrent neural networks (RNNs) [3], convolutional neural networks (CNNs) [6], and Transformer [7] to effectively capture complex patterns and dependencies in system logs. Despite these advantages, challenges still present, such as high-dimensional and

noisy log data, class imbalance, generalization across datasets, and the interpretability of models.

Recently, large language models (LLMs), such as ChatGPT [8], have shown promising results in many domains such as language understanding [9], dialogue [10] and machine translation [11]. Liu et al. [12] leverage ChatGPT to recommend products for users. Cheng et al. [8] used GPT-4 to perform end-to-end data analysis with databases from a wide range of domains. Le et al. [13] leverage ChatGPT as a log parser to extract the log event and parameters. Nonetheless, log-based anomaly detection with ChatGPT has not been thoroughly investigated. To the best of our knowledge, this is the first study on how to use ChatGPT in log-based anomaly detection.

In this work, we propose LogGPT, a log-based anomaly detection framework based on ChatGPT, which consists of three components: *log preprocessing*, *prompt construction*, and *response parser*. The objective of LogGPT is to utilize ChatGPT’s ability on language understanding from large-scale corpora in the domain of system log analysis. The *log preprocessing* component involves filtering, parsing, and grouping to transform raw log messages into a structured format for further analysis. *Prompt construction* focuses on designing specific prompts tailored to log anomaly detection, aiming to instruct ChatGPT’s generation process toward accurate anomaly identification. The *response parser* is responsible for extracting the output returned by ChatGPT, allowing for further analysis and evaluation of the detected anomalies.

LogGPT utilizes its language generation capabilities for log anomaly detection. We investigate the possibility to transfer the knowledge and patterns learned by ChatGPT from diverse textual sources to the specialized domain of system log analysis, enabling the effective detection of abnormal events. By conducting comprehensive experiments and analyses, we aim to gain a deeper understanding of the potential and limitations of ChatGPT for log-based anomaly detection. Particularly, we focus on answering the following research questions: 1) *What is the current capability of ChatGPT for log-based anomaly detection?* and 2) *How explainable are the anomalies detected by the model?*

To answer the above questions, we conduct a systematic evaluation of LogGPT and compare it to three deep learning-based methods (including DeepLog [3], LogAnomaly [4], LogRobust [14]) on two datasets (BGL and Spirit [15]), under controlled experimental settings. We first investigate the impact of different variables (including prompt construction and window size) on LogGPT performance. Then, we compare the performance of LogGPT to three baseline methods. Finally,

Jiaying Qi, Shaohan Huang, Zhongzhi Luan, Hailong Yang, and Depei Qian are with the Sino-German Joint Software Institute, Beihang University, Beijing, 100191, China.

E-mail: {jiayingqi, luan.zhongzhi}@buaa.edu.cn.

Carol Fung is with the Concordia Institute for Information Systems Engineering, Concordia University, Quebec, Canada.

(Zhongzhi Luan is the corresponding author for this work.)

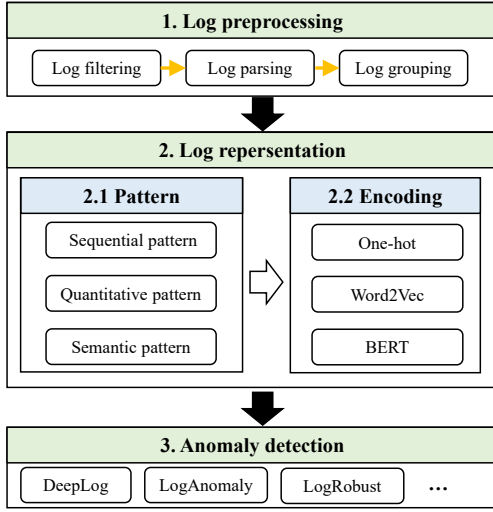


Fig. 1. Log-based anomaly detection workflow.

we explore the interpretability of LogGPT. Through extensive experiments, we obtained the following major findings about the LogGPT for log-based anomaly detection:

- The prompt construction (both task description and human knowledge injection) has a significant impact on LogGPT. A more specific task description and injecting normal log information are often beneficial.
- The window size affects the performance of LogGPT. Increasing the window size usually results in better performance.
- Compared to three deep learning-based methods, LogGPT shows promising performance (zero-shot and few-shot) on both the BGL and Spirit datasets.
- LogGPT demonstrates excellent interpretability in detecting anomalies, providing users with specific information to aid in understanding the causes of anomalies and offering potential preventive suggestions.

In summary, the major contributions of this work are as follows:

- This study represents the first attempt to employ ChatGPT for log-based anomaly detection and provides a quantitative evaluation of its effectiveness.
- We designed LogGPT, a common framework specifically designed for log-based anomaly detection. The LogGPT framework combines three components to identify and analyze anomalies within log data.
- We conducted extensive experiments on both BGL and Spirit datasets and demonstrated that LogGPT has promising performance and good interpretability.

II. PRELIMINARY

The common workflow of log-based anomaly detection is shown in Figure 1, which includes three steps: 1) log preprocessing, 2) log representation, 3) anomaly detection through DL models.

1) *Log preprocessing*: Log messages are semi-structured texts, which consist of a constant part (log event) and a variable part (parameters). Generally, we require to filter, parse, and group raw logs to train an anomaly detection model. First, some noise is removed by log filtering. Then, log events and parameters are automatically extracted using log parsing methods. For example, many log parsing methods have been proposed, such as Drain [16], Spell [17], and Paddy [18]. Finally, logs will be separated into various groups, where each group contains several log records. These groups are called log sequences, which will extract various patterns as the input of anomaly detection models.

2) *Log representation*: Log sequences necessitate transformation into feature vectors for utilization as input of DL models. There are three primary types of log patterns: **Sequential pattern**, which represents the contextual information of log sequences. **Quantitative pattern**, which statistic each log event occurs distribution within log sequences. **Semantic pattern**, which represents the semantic meaning of each log event using a language model, aims to extract the associated semantic information of log sequences. In order to represent the aforementioned log patterns in the form of a feature vector, commonly employed encoding methods include one-hot encoding, word2vec, and BERT [15].

3) *Anomaly detection*: The main purpose of this step is to train a deep anomaly detection model with input as feature vectors of log sequences. A variety of DL techniques have been applied to log-based anomaly detection, such as CNN, RNN, and Transformer [15]. These models can be grouped into three types based on training strategy: supervised, semi-supervised, and unsupervised. Supervised models consider log-based anomaly detection as a binary classification task, which utilizes both normal and abnormal logs in the training stage. Semi-supervised models capture normal patterns from the normal log sequences to detect anomalies. Unsupervised models do not require any labeled logs and typically combine with cluster and generative methods.

III. LOG-BASED ANOMALY DETECTION WITH CHATGPT

We design a framework, namely LogGPT, for log-based anomaly detection using ChatGPT. As shown in Figure 2, the framework consists of three main components: **Log preprocessing**, where the raw log messages are parsed into a structured format; **Prompt construction**, where different anomaly detection prompts are constructed for log sequences; and **Response parser**, where prompts and sequences will form a request to be sent to ChatGPT. The response information will be parsed into parts for evaluation, and the final result will be presented to the users.

A. Log Preprocessing

This step is to extract the structured information from raw logs. We employed the state-of-the-art log parsing method (Drain) to extract structured data, including *ID*, *Timestamp*, *Content*, and *EventTemplate*. After that, the raw logs and parsed logs are grouped into different chunks using a fixed-size

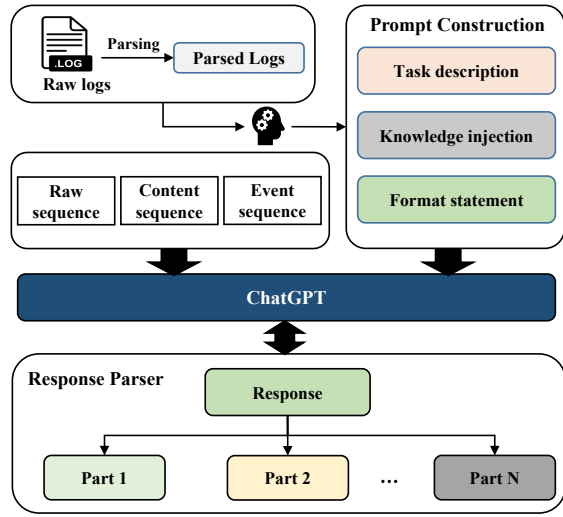


Fig. 2. The framework of LogGPT to perform log-based anomaly detection.

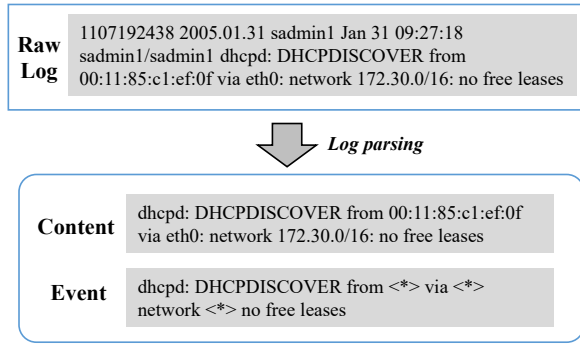


Fig. 3. An example of log parsing.

time window, which generates three types of sequences: *raw sequence*, *content sequence*, and *event sequence*. An example of this step is shown in Figure 3.

The *raw sequence* consists of the raw log messages, capturing the unaltered information directly from the log messages. The *content sequence* focuses on the log text. It excludes certain content that is irrelevant to the analysis, such as *ID* and *Timestamp*. The event sequence is the high-level abstraction of the *content sequence* where dropped the variable part in the log text. Compared to the three types of sequence, the *raw sequence* provides fine-grained information, enabling detailed investigation and troubleshooting. The *content sequence* emphasizes the textual content, facilitating text-based analysis and anomaly detection. The event sequence provides coarse-grained information, highlighting sequential patterns to support pattern-based anomaly detection.

B. Prompt Construction

In this step, we describe the prompt construction strategy for the log-based anomaly detection task. As shown in Figure 4, we designed a prompt template, which includes the following parts: *task description*, *format statement*, *human knowledge*

injection, *input sequence*. Then, we fill in the content of each part by domain experience and improve the prompt using ChatGPT. In addition, we follow the general tips that include more specific, starting simple and iterating on improvements, for designing task-specific prompts. Finally, we selected two prompts with the best test performance to conduct our experiments.

1) *Task description*. Log-based anomaly detection is a crucial task for maintaining system operations. While DL-based methods have made some progress in this field, their interpretability remains a challenging issue [5]. Therefore, the *task description* should not only instruct ChatGPT to determine whether an anomaly has occurred but also prompt it to provide explanations for the occurrence of anomalies. Furthermore, it should guide ChatGPT to suggest possible preventive measures. To achieve these objectives, we introduce two types of instruction in this part, namely indirect instruction and direct instruction. For indirect instruction, we use a phrase, such as "Output your thought process", to prompt ChatGPT to explain the anomalous events. For direct instruction, we explicitly instruct ChatGPT to generate anomaly reports and preventive measures. This way, we aim to provide users with more fine-grained system reports and help users achieve a deeper understanding of the operation status of the system.

2) *Format statement*. To ensure response diversity, ChatGPT incorporates some randomness during the generation process. Additionally, when users sent requests to ChatGPT, they can specify a temperature parameter to control the diversity of the response. Higher temperature values make ChatGPT prefer to select words and phrases more randomly, resulting in more diverse and creative text generation [12]. However, higher temperature values may also cause the model to choose less common or less reasonable vocabulary, leading to unexpected responses. While such responses can be more creative, they also lead to inaccurate or unreasonable outcomes, making it challenging to evaluate the performance of anomaly detection. To address this concern, we introduce two methods to handle this challenge. First, we explicitly indicate the expected response format in the prompt. For example, we indicate the response must be in json format and specify the keys that should be included, which ensures that most of the responses meet the expected format (*Output format: Please note that return back in following json format, include keys: is_anomaly, reports, preventive_measures*). However, there may still be some responses that do not meet the expectations. We introduce the *Response Parser* component (see Section III-C) to address this issue.

3) *Human Knowledge Injection*. DL-based methods have demonstrated that the performance is often unsatisfactory without any domain prior knowledge (unsupervised methods) [5]. Therefore, it should be beneficial to introduce some prior knowledge in the prompt. We have introduced this part (optional, gray) to the prompt template to allow users to inject specific domain prior knowledge into the prompt, which aims to improve the performance of ChatGPT on log anomaly detection tasks. Generally, we refer to this kind of prompt

Prompt template	<Task description> \n <Format statement> \n <Human knowledge injection> \n <Log sequence> \n Output: \n
Prompt-1	Your task is to determine if a given set of log messages contains an anomaly or not (Sorted by timestamp).\n Here are some examples of one log: {placeholder} \n Based on above examples, please determine the following logs contains an anomaly or not. Be careful to consider the contextual information of the log sequence.\n Use the following format:\n Logs: Given a set of log meassages here. (a python list)\n Answer:\n yes or no (Output your thought process) \n Log: {placeholder} \n Answer: \n
Prompt-2	Your task is to determine whether a given set of logs contains an anomaly or not and provide a binary classification: 0 for normal and 1 for the anomaly. If an anomaly, you should generate reports and suggest relevant preventive measures. Note that do not a lot of text.\n Output format: Return back in json format, including keys: is_anomaly, reports, preventive_measures.\n Here are some normal (abnormal) logs:\n {placeholder} \n Input log data (a python list): {placeholder} \n Output: \n

Fig. 4. Example prompts of log-based anomaly detection. For the zero-shot setting, the human-knowledge injection part is dropped.

as a few-shot setting, otherwise, it is a zero-shot setting. For example, we can use some labeled logs to fill this part, so that ChatGPT can preview some domain knowledge.

4) *Input sequence*. There are three types of input sequences mentioned above. To this end, we provide a variable part (blue) that supports different input sequences. When sending a request to ChatGPT, we form the log sequence into a Python list and concatenate other parts as a final prompt.

C. Response Parser

The responses of ChatGPT have diversity. In addition, it should be noted that the output token of ChatGPT has a maximum length limit for each request. Once the limit is exceeded, the response will be terminated and the current output will be returned. To ensure the parsability of the response, we designed this component to parse the text of the response into several pre-defined parts. For example, three parts are defined in this study including *is_anomaly*, *reports*, and *preventive_measures*. For each response, we format and check the response text. If the pre-defined format is satisfied, the text is parsed directly. Otherwise, we use ChatGPT to reformat the text so that it satisfies the requirements.

IV. EVALUATION

In this section, we evaluate LogGPT by answering the following research questions:

- **RQ1:** How does LogGPT perform with different prompts?
- **RQ2:** How does LogGPT perform on different window sizes?
- **RQ3:** How does human knowledge injection affect the performance of LogGPT?
- **RQ4:** How does LogGPT perform compared to deep learning-based methods?
- **RQ5:** How explainable are the anomalies detected by LogGPT?

A. Datasets

Our performance evaluation process is based on two commonly used log datasets [15], and the details of each dataset are as follows:

- **Blue Gene/L (BGL)** dataset contains 4,747,963 log messages from a Blue Gene/L supercomputer system at Lawrence Livermore National Laboratory, California. It has 131,072 processors and 32,768GB of memory. The dataset includes alert and non-alert messages, with 348,460 (7.34%) labeled as anomalous.
- **Spirit** dataset is collected from a Linux production cluster at Sandia National Labs, comprising 512 nodes over a 23-day period. It contains 272,298,969 log messages, with 172,816,564 (63.47%) labeled as system anomalies.

We split the training and testing set with 8:2. Due to the imposed limitations on ChatGPT's API request frequency, we adopted a random sampling strategy to select a subset consist 2000 consecutive logs from the testing set. Moreover, we have checked manually to ensure a proportion of abnormal logs within the subset. This approach aimed to sample a representative subset of logs for evaluation.

B. Baseline methods and Metrics

- **Deeplog** [3] is a semi-supervised method that uses sequential vectors as input patterns to learn normal system executions by predicting the next log event based on preceding events.
- **LogAnomaly** [4] is a semi-supervised method that uses sequential vector and quantitative vector as input, as well as applies an LSTM-based model to detect sequential and quantitative anomalies in log sequence.
- **LogRobust** [14] is a supervised log-based anomaly detection method that extracts semantic information of log events and represents them as semantic vectors. It then uses an attention-based Bi-LSTM model to detect anomalies, which can capture contextual information in log sequences and have better robust for log unstable.

Evaluation Metrics. Like previous works [15], [19], we use common evaluation metrics for anomaly detection tasks, including F1 score, Precision, and Recall. In addition, Specificity is also used in order to evaluate more comprehensively.

C. Implementation details

To extract the contents and events from the log data, we use the log parser *Drain* with the default parameter settings [5]. Note that the special character ($\langle * \rangle$) in log events is dropped when grouped into event sequences. For the ChatGPT, we apply *gpt-3.5-turbo*¹ to conduct zero-shot and few-shot experiments with two prompts (Figure 4). We set the temperature is 0 and only the top-1 choice is returned. Moreover, the maximum number of output tokens is limited to 100 for a faster response. In the few-shot setting, we put the 5 historical logs and labels to enable ChatGPT to learn some prior knowledge. For the response parser, we designed a prompt to reformat the response in *json* format if the pre-defined format is not satisfied: "Please format the following text in json format, which include the keys: ... ". The implementation of three baseline methods is referred to the public code on GitHub².

D. Results and Analysis

1) RQ1: Performance with different prompts:

Experiment Settings. We investigated the performance of LogGPT with different prompts. Specifically, we used both prompt-1 (P1) and prompt-2 (P2) as shown in Figure 4, and evaluated the performance of the three types of input sequences in both zero-shot (ZS) and few-shot (FS) settings, with a fixed window size of 50. The experimental results are presented in Figure 5.

We can see that for the raw sequence, P1 has a higher F1 score than P2. In particular, it can be seen that human knowledge injection is not always beneficial, since the ZS-P1 with the raw sequence is the highest in terms of F1 score on both datasets. P2 has more advantages than P1 for content sequence and event sequence. For example, FS-P2 has an F1 score of 0.694 on Spirit, which is much higher than other types of inputs. Comparing the three different sequence types, the content sequence performs better than the other two sequence types. We believe this is because the content removes irrelevant information (such as timestamps), which allows ChatGPT to understand its semantic information more precisely.

Summary. The prompts have a significant impact on the log-based anomaly detection task. In contrast to expectations, the incorporation of human knowledge does not always result in improved detection performance. It is essential to explore various prompts that align with the specific context in order to attain satisfactory detection performance.

2) RQ2: Performance on different window sizes:

Experiment Settings. We evaluated the performance of LogGPT with the window size increasing from 10 to 50. P2 is the default prompt. The experimental results are presented in Figure 6 and more results are presented in Appendix VIII.

TABLE I
PERFORMANCE WITH DIFFERENT INJECTION TYPES ON THE SPIRIT DATASET

Window Size	Injection Type	F1	P	R	S
10	Abnormal	0.026	0.100	0.015	0.932
20		0.000	0.000	0.000	0.000
30		0.490	0.545	0.444	0.750
40		0.000	0.000	0.000	0.000
50		0.485	0.500	0.471	0.652
10	Normal	0.601	0.432	0.985	0.333
20		0.629	0.459	1.000	0.246
30		0.740	0.587	1.000	0.525
40		0.714	0.556	1.000	0.467
50		0.694	0.531	1.000	0.348

We can observe a positive correlation between the F1 score and window size in the majority of cases, indicating that larger window sizes generally result in higher F1 scores. This suggests that incorporating more contextual information is often beneficial. Furthermore, the three types of sequences can be ranked in terms of performance from best to worst as follows: content sequence, event sequence, and raw sequence. This ranking aligns with our expectations, as the raw sequence contains a significant amount of irrelevant information, while the event sequence lacks parameter information. Finally, the results also demonstrate the advantages of the few-shot setting in most cases.

Summary. The window size affects the performance of LogGPT where increasing the window size usually results in higher accuracy in anomaly detection. In general, better performance could be achieved through the content sequence and human knowledge injection.

3) RQ3: The impact of human knowledge injection:

Experiment Settings. We studied the effects of different human knowledge injections. Two types of knowledge injection (abnormal and normal) are employed, we randomly sample 5 normal logs and 5 abnormal logs from the Spirit dataset. The experimental results are presented in Table II.

We can observe that the performance of LogGPT varies a lot with different types of knowledge injection. For example, abnormal knowledge injection results in an F1 score range of [0.000, 0.490] with the window size increasing from 10 to 50. When experimenting with the normal knowledge injection, it achieves a promising performance with the F1 scores ranging from 0.601 to 0.740. Because the abnormal type is diverse, a few abnormal samples cannot cover most types. The results also show that the recall remains high across all window sizes, while precision and specificity are low. This suggests that LogGPT frequently misses identical normal sequences as abnormal, resulting in a high false positive rate.

Summary. Different human knowledge injections have a significant impact on the performance of log-based anomaly detection. LogGPT adopts a conservative approach by identifying only highly certain logs as normal and identifying the remaining logs as abnormal. This bias can be attributed to the presence of many low-level alert events in the logs, which are

¹<https://platform.openai.com/docs/models/overview>

²<https://github.com/LogIntelligence/LogADEmpirical>

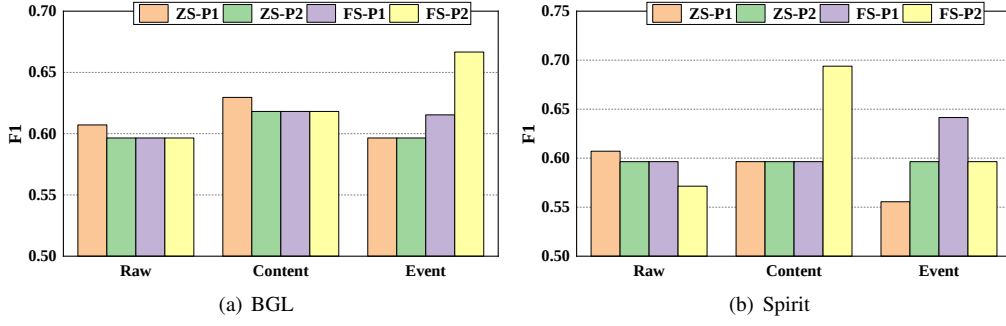


Fig. 5. Performance of LogGPT with different prompts.

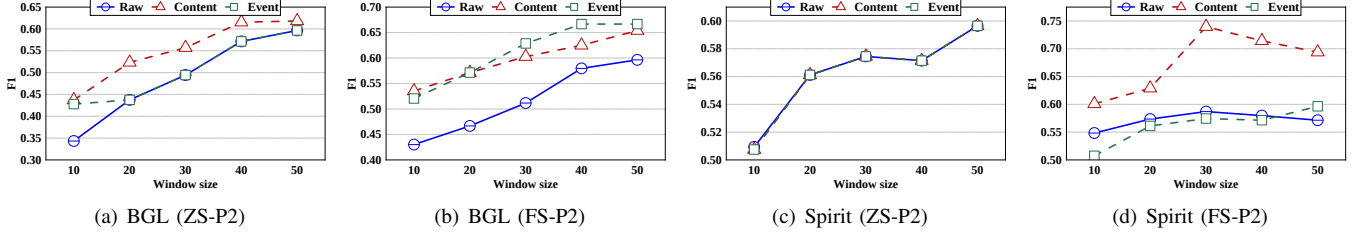


Fig. 6. Performance of LogGPT w.r.t different window size.

TABLE II
PERFORMANCE COMPARISON ON BGL AND SPIRIT (PROMPT-2 WITH CONTENT SEQUENCE)

Window Size	Metrics	BGL					Spirit				
		DeepLog	LogAnomaly	LogRobust	LogGPT (zero-shot)	LogGPT (few-shot)	DeepLog	LogAnomaly	LogRobust	LogGPT (zero-shot)	LogGPT (few-shot)
10	F	0.168	0.201	0.944	0.437	0.444	0.521	0.679	0.667	0.507	0.601
	P	0.920	0.112	0.975	0.280	0.286	0.352	0.602	1.000	0.340	0.432
	R	0.992	0.985	0.914	1.000	1.000	1.000	0.779	0.500	1.000	0.985
	S	0.370	0.501	0.999	0.356	0.375	0.053	0.735	1.000	0.000	0.333
20	F	0.219	0.228	0.601	0.523	0.523	0.627	0.614	0.700	0.561	0.629
	P	0.123	0.129	0.442	0.354	0.354	0.591	0.500	1.000	0.390	0.459
	R	0.977	0.975	0.938	1.000	1.000	0.667	0.795	0.538	1.000	1.000
	S	0.513	0.552	0.919	0.292	0.292	0.705	0.492	1.000	0.000	0.246
30	F	0.129	0.146	0.891	0.557	0.571	0.581	0.632	0.744	0.574	0.740
	P	0.688	0.788	0.893	0.386	0.400	0.409	0.600	1.000	0.403	0.587
	R	1.000	0.972	0.888	1.000	1.000	1.000	0.667	0.593	1.000	1.000
	S	0.082	0.180	0.992	0.222	0.267	0.025	0.700	1.000	0.000	0.525
40	F	0.147	0.239	0.374	0.615	0.625	0.597	0.711	0.667	0.571	0.714
	P	0.793	0.137	0.236	0.444	0.455	0.426	0.640	1.000	0.400	0.556
	R	0.972	0.960	0.900	1.000	1.000	1.000	0.800	0.500	1.000	1.000
	S	0.137	0.541	0.780	0.167	0.200	0.100	0.700	1.000	0.000	0.467
50	F	0.224	0.243	0.304	0.618	0.618	0.607	0.571	0.455	0.596	0.694
	P	0.126	0.139	0.183	0.447	0.447	0.436	0.410	1.000	0.425	0.531
	R	0.994	0.942	0.911	1.000	1.000	1.000	0.941	0.294	1.000	1.000
	S	0.418	0.537	0.676	0.087	0.087	0.043	0.000	1.000	0.000	0.348

often disregarded by administrators despite being semantically categorized as abnormal.

4) RQ4: Performance comparison to three baseline methods:

Experiment Settings. We evaluated the performance of LogGPT with both zero-shot setting and few-shot setting and used prompt-2 as the default prompt. The experimental results are shown in Table III.

Compared to the three deep learning-based methods, LogGPT with few-shot setting achieved superior performance on both datasets in terms of the F1 score, precision, and recall. And LogGPT with the zero-shot setting achieved a competitive

F1 score on both datasets. An interesting observation is that LogGPT achieved the same F1 score between zero-shot and few-shot settings on BGL when the window size is 20 and 50, suggesting that adding some prior knowledge is not always beneficial. Furthermore, LogGPT achieves the highest recall, but the specificity is underperforming, especially on Spirit where the zero-shot setting is 0. This demonstrated that LogGPT is more reserved, resulting in the identification of many normal logs as anomalies.

Summary. LogGPT shows promising performance on both the BGL and Spirit datasets. Its ability to identify log sequences as normal or anomalous suggests its potential as an

TABLE III
INTERPRETABILITY OF DETECTED ANOMALIES ON BGL AND SPIRIT

Dataset	Key	Response
BGL	Anomaly	...
	log sequence	RAS KERNEL FATAL data TLB error interrupt
		...
	Reports	data TLB error interrupt
	Preventive measures	Check the memory and CPU usage Check the system logs
BGL	Anomaly	...
	log sequence	RAS KERNEL INFO MACHINE CHECK DCR read timeout
		...
	Reports	RAS KERNEL FATAL Lustre mount FAILED : bglio78 : block_id : location
	Preventive measures	Check the Lustre mount FAILED
Spirit	Anomaly	...
	log sequence	pbs_mom: Connection refused (111) in open_demux, open_demux: connect 172.30.71.32:44020
		...
	Reports	Anomaly detected: Multiple sshd sessions opened for user root
	Preventive measures	Limit the number of sshd sessions opened for user root
Spirit	Anomaly	...
	log sequence	pbs_mom: Connection refused (111) in open_demux, open_demux: connect 172.30.71.32:43908
		...
	Reports	Inappropriate ioctl for device (25) in search_env_and_open, failed connect to mpiexec process on MS
	Preventive measures	Check the mpiexec process on MS

effective tool for log anomaly detection in various domains.

5) *RQ5: Interpretability of anomalies by LogGPT:*

Providing more explanations for the anomalies detected by the models has been difficult since DL-based methods are considered a black box whose decision-making process is not explainable, which poses challenges for administrators in timely identifying and preventing anomalies. In this experiment, we investigate the potential of LogGPT for interpretability with respect to anomaly localization and prevention.

Experiment Settings. As LogGPT(few-shot) achieved the highest recall on BGL and Spirit datasets, we randomly selected two anomalous raw sequences on both datasets, respectively, to verify the effectiveness of anomaly localization and anomaly prevention.

The results are presented in Tabel III. We can see that the response of LogGPT can not only report the cause of the anomaly but also give suggestions for prevention. For example, an anomaly occurred which is *connection refused* during the running of a Linux cluster. LogGPT reports the reason to be the root user opening too many *sshd sessions*. Furthermore, LogGPT suggests to enforce the limitations on the number of *sshd sessions* the root user can open. However, LogGPT’s suggestions may not always be effective and they occasionally lead to confusion. For example, in the case of the first example on the BGL dataset, LogGPT suggests that users should first check the CPU and memory usage.

If the anomaly persists, LogGPT suggests the administrators to manually analyze historical logs. We can see from these results that while LogGPT is a powerful log-based anomaly detector, its suggestions are generated based on patterns and information extracted from the input sequences. Therefore, there are instances where its suggestions may not be useful or may require additional manual analysis by administrators.

Summary. LogGPT can help administrators find abnormal locations and provide preventive suggestions which improve the efficiency of troubleshooting. However, there are some ineffective outputs in our experiments due to the hallucination problem of ChatGPT. Since this work is not concerned with solving this problem, we only report the existence of ineffective output. We think more specific prompts may be able to reduce such output.

V. RELATED WORKS

In this section, we briefly review the related work of log-based anomaly detection and large language models (LLMs).

Log-based anomaly detection is an important technique for monitoring system activities and identifying suspicious behaviors. Many works have been proposed in recent years [3], [4], [14], these works can be grouped into three types according to the training strategy.

1) **Supervised methods** assume that have a large number of labeled data to train binary classification models [14], which commonly achieved the optimal performance. Zhou et.al [20]

proposed LogSayer, which is a log pattern-driven anomaly detection model that addresses these challenges. LogSayer utilizes statistical features, LSTM neural networks, and a BP neural network for adaptive anomaly decisions. Du et al. [21] proposed LogAttention which embeds log patterns into semantic vectors and uses a self-attention-based neural network to detect anomalies. The input to these methods typically is a log event sequence. However, the parameters within the logs also contain valuable information for identifying anomalies. Huang et al. [7] proposed HitAnomaly, which utilizes a hierarchical transformer model to capture both log event sequences and parameters, to improve the detection accuracy.

2) **Semi-supervised methods** assume that training takes place only on normal data that is free of anomalies. Its primary idea is to make the models capture the normal sequence patterns, while the abnormal sequence patterns are quite distinct. Some works focus on reconstruction-based methods that utilize generative models, such as AutoEncoders (AEs) [22] and Generative Adversarial Networks (GANs) [23], to encode the input log sequences and then attempt to reconstruct the input using a decoder. Any input data that is fed into an already trained model and yields a high reconstruction error is then considered anomalous. Another technical route is using sequential models, such as Recursive Neural Networks (RNNs) [3] and Transformer [24], to learn normal sequential patterns and predict the next possible log event. If a new log sequence does not match the predicted then it is identified as anomalous. Moreover, we note that many works [22] claim that it is unsupervised, but it is actually trained using only normal logs.

3) **Unsupervised methods** [25], [26] assume that no labels are available for training models. Han et al. [27] proposed a domain adaptation framework called LogTAD, which makes log data from different systems have similar distributions, enabling the detection model to identify anomalies across multiple systems. Nedelkoski et al. [28] focus on learning log representations that capture semantic differences between normal and anomaly logs, improving generalization on unseen logs. It leverages auxiliary log datasets available on the internet to enhance the representation of normal data while providing diversity to prevent overfitting. However, due to the lack of prior knowledge, the detection accuracy is lower than supervised and semi-supervised methods in most scenarios.

Large language models (LLMs) have revolutionized the field of natural language processing (NLP) and have gained significant attention in recent years. OpenAI introduced GPT-1 [29], which demonstrated promising results by pre-training on a large corpus of internet text data and fine-tuning it for specific tasks. After that, most LLMs have been proposed, such as BERT [24], T5 [30], and GPT-3 [31], etc. A key milestone in LLM development is InstructGPT [32], which introduces a framework for instruction fine-tuning of pre-trained language models using Reinforcement Learning from Human Feedback (RLHF). This framework enables LLMs to adapt to a wide range of NLP tasks, enhancing their versatility and flexibility by incorporating human feedback. Unlike LLMs trained alone

on text corpora through unsupervised pre-training, RLHF allows models to align with human preferences and values, significantly improving their performance. OpenAI has leveraged similar techniques to develop ChatGPT, a conversation-based language model that brings AI to the forefront, transitioning it from being behind the scenes. Subsequently, OpenAI released a version based on GPT-4 [33]. Concurrently, other peers have also released similar language models. For example, Meta AI's LLaMA [34] and Google's PaLM [35], etc.

In terms of applications, LLMs have shown impressive performance in various domains. For example, Microsoft released the Copilot plugin [36], a dedicated model for code generation, which greatly improves the efficiency of developers. Additionally, such models as Stable Diffusion, Midjourney, and DALL-E [37] have also demonstrated excellent results in artificial intelligence generative content (AIGC). We can envision that LLMs will play a crucial role in numerous decision-making scenarios, paving the way for advancements in various domains.

VI. LIMITATIONS

Based on our observations, we highlight several limitations of LogGPT:

- **Sensitivity to prompt variation:** Designing the optimal prompt is not always straightforward, even though prompts play a critical role in LogGPT's performance.
- **Limited window size:** Balancing the need for a larger window size with computational constraints remains a challenge.
- **High false positive rate:** LogGPT suffers from a high false positive rate, leading to a significant number of incorrect anomaly identifications.
- **Trustworthiness:** The presence of ineffective outputs due to the hallucination problem of ChatGPT.

Addressing these limitations will improve the applicability and effectiveness of LogGPT, and benefit the application of LogGPT in real-world scenarios.

VII. CONCLUSION AND FUTURE WORK

In this study, we proposed LogGPT, a framework for log-based anomaly detection based on ChatGPT. Experiments demonstrated that LogGPT has a promising ability for this important task and better anomalous interpretability, except that many normal sequences are identified anomalies and it causes many false alarms. Moreover, we investigated different prompts construction, window sizes, and the types of input sequence impact on performance. We believe that the results and findings of our study can provide valuable insights into the strengths and limitations of ChatGPT in log-based anomaly detection. As our future work, we will investigate the following:

- Experiment with more public log datasets and test more prompts to improve performance.
- Try using more large language models such as GPT-4 or LLaMA.

- Further exploration of ChatGPT’s potential for anomalous interpretability.

REFERENCES

- [1] J. Breier and J. Branišová, “A dynamic rule creation based anomaly detection method for identifying security breaches in log records,” *Wireless Personal Communications*, vol. 94, pp. 497–511, 2017.
- [2] J. Breier and J. Braniš, “Anomaly detection from log files using data mining techniques,” in *Information Science and Applications*. Springer, 2015, pp. 449–457.
- [3] M. Du, F. Li, G. Zheng, and V. Srikumar, “Deeplog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 1285–1298.
- [4] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun *et al.*, “Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs,” in *IJCAI*, vol. 19, no. 7, 2019, pp. 4739–4745.
- [5] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” *ACM computing surveys (CSUR)*, vol. 54, no. 6, pp. 1–37, 2021.
- [6] S. Lu, X. Wei, Y. Li, and L. Wang, “Detecting anomaly in big data system logs using convolutional neural network,” in *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*. IEEE, 2018, pp. 151–158.
- [7] S. Huang, Y. Liu, C. Fung, R. He, Y. Zhao, H. Yang, and Z. Luan, “Hitanomaly: Hierarchical transformers for anomaly detection in system log,” *IEEE Transactions on Network and Service Management*, vol. 17, no. 4, pp. 2064–2076, 2020.
- [8] L. Cheng, X. Li, and L. Bing, “Is gpt-4 a good data analyst?” *arXiv preprint arXiv:2305.15038*, 2023.
- [9] S. Frieder, L. Pinchetti, R.-R. Griffiths, T. Salvatori, T. Lukasiewicz, P. C. Petersen, A. Chevalier, and J. Berner, “Mathematical capabilities of chatgpt,” *arXiv preprint arXiv:2301.13867*, 2023.
- [10] N. Chen, Y. Wang, H. Jiang, D. Cai, Z. Chen, and J. Li, “What would harry say? building dialogue agents for characters in a story,” *arXiv preprint arXiv:2211.06869*, 2022.
- [11] W. Jiao, W. Wang, J.-t. Huang, X. Wang, and Z. Tu, “Is chatgpt a good translator? a preliminary study,” *arXiv preprint arXiv:2301.08745*, 2023.
- [12] J. Liu, C. Liu, R. Lv, K. Zhou, and Y. Zhang, “Is chatgpt a good recommender? a preliminary study,” *arXiv preprint arXiv:2304.10149*, 2023.
- [13] V.-H. Le and H. Zhang, “An evaluation of log parsing with chatgpt,” *arXiv preprint arXiv:2306.01590*, 2023.
- [14] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li *et al.*, “Robust log-based anomaly detection on unstable log data,” in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 807–817.
- [15] V.-H. Le and H. Zhang, “Log-based anomaly detection with deep learning: How far are we?” in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1356–1367.
- [16] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “Drain: An online log parsing approach with fixed depth tree,” in *2017 IEEE international conference on web services (ICWS)*. IEEE, 2017, pp. 33–40.
- [17] M. Du and F. Li, “Spell: Streaming parsing of system event logs,” in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, 2016, pp. 859–864.
- [18] S. Huang, Y. Liu, C. Fung, R. He, Y. Zhao, H. Yang, and Z. Luan, “Paddy: An event log parsing approach using dynamic dictionary,” in *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020, pp. 1–8.
- [19] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Zhang, “Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning,” in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 623–634.
- [20] P. Zhou, Y. Wang, Z. Li, X. Wang, G. Tyson, and G. Xie, “Logsayer: Log pattern-driven cloud component anomaly diagnosis with machine learning,” in *2020 IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)*. IEEE, 2020, pp. 1–10.
- [21] Q. Du, L. Zhao, J. Xu, Y. Han, and S. Zhang, “Log-based anomaly detection with multi-head scaled dot-product attention mechanism,” in *Database and Expert Systems Applications: 32nd International Conference, DEXA 2021, Virtual Event, September 27–30, 2021, Proceedings, Part I 32*. Springer, 2021, pp. 335–347.
- [22] L. Zhang, W. Li, Z. Zhang, Q. Lu, C. Hou, P. Hu, T. Gui, and S. Lu, “Logattn: Unsupervised log anomaly detection with an autoencoder based attention mechanism,” in *Knowledge Science, Engineering and Management: 14th International Conference, KSEM 2021, Tokyo, Japan, August 14–16, 2021, Proceedings, Part III*. Springer, 2021, pp. 222–235.
- [23] Z. Zhao, W. Niu, X. Zhang, R. Zhang, Z. Yu, and C. Huang, “Trine: Syslog anomaly detection with three transformer encoders in one generative adversarial network,” *Applied Intelligence*, vol. 52, no. 8, pp. 8810–8819, 2022.
- [24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [25] K. Otomo, S. Kobayashi, K. Fukuda, and H. Esaki, “Latent variable based anomaly detection in network system logs,” *IEICE TRANSACTIONS on Information and Systems*, vol. 102, no. 9, pp. 1644–1652, 2019.
- [26] S. Bursic, V. Cuculo, and A. D’Amelio, “Anomaly detection from log files using unsupervised deep learning,” in *Formal Methods. FM 2019 International Workshops: Porto, Portugal, October 7–11, 2019, Revised Selected Papers, Part I 3*. Springer, 2020, pp. 200–207.
- [27] X. Han and S. Yuan, “Unsupervised cross-system log anomaly detection via domain adaptation,” in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 2021, pp. 3068–3072.
- [28] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, “Self-attentive classification-based anomaly detection in unstructured logs,” in *2020 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2020, pp. 1196–1201.
- [29] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, “Improving language understanding by generative pre-training,” 2018.
- [30] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [31] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [32] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [33] OpenAI, “Gpt-4 technical report,” 2023.
- [34] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [35] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, “Palm: Scaling language modeling with pathways,” *arXiv preprint arXiv:2204.02311*, 2022.
- [36] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [37] A. Borji, “Generated faces in the wild: Quantitative comparison of stable diffusion, midjourney and dall-e 2,” *arXiv preprint arXiv:2210.00586*, 2022.

VIII. APPENDIX

More experimental results on both BGL and Spirit datasets are shown in Figure 7 and Figure 8. The window size is increased from 10 to 50 and all three types of input sequences are evaluated. The *gpt-3.5-turbo* model is used in our experiments. Our source code and detailed experimental data will be made available with the publication of this work.

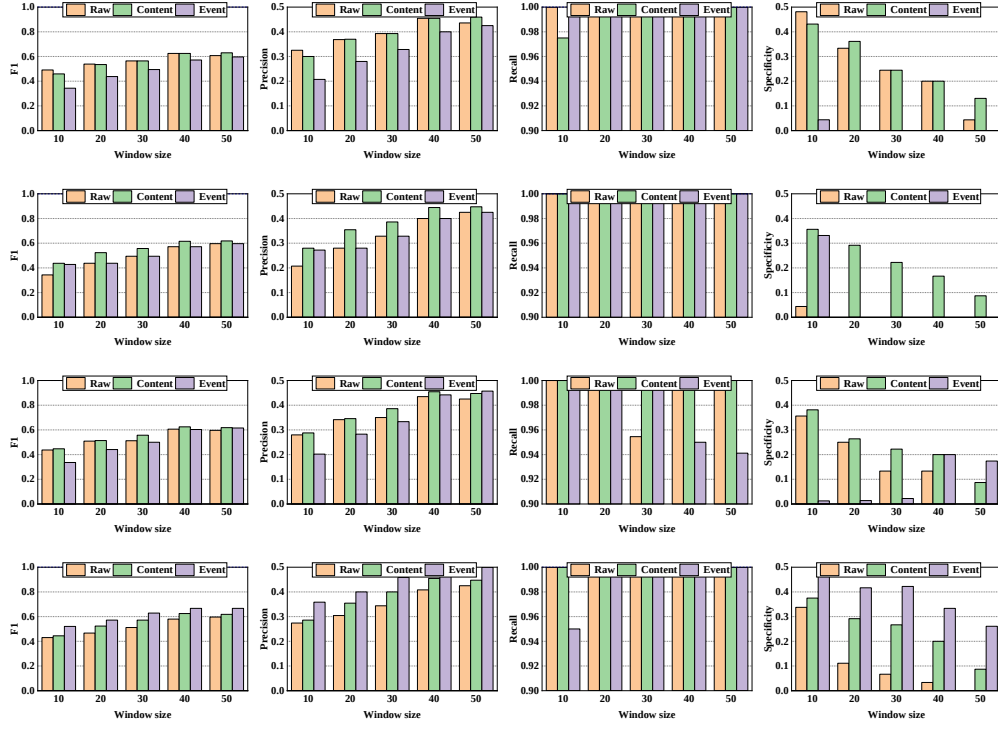


Fig. 7. BGL. First row: Prompt-1 (zero-shot); Second row: Prompt-2 (zero-shot); Third row: Prompt-1 (few-shot); Last row: Prompt-2 (few-shot).

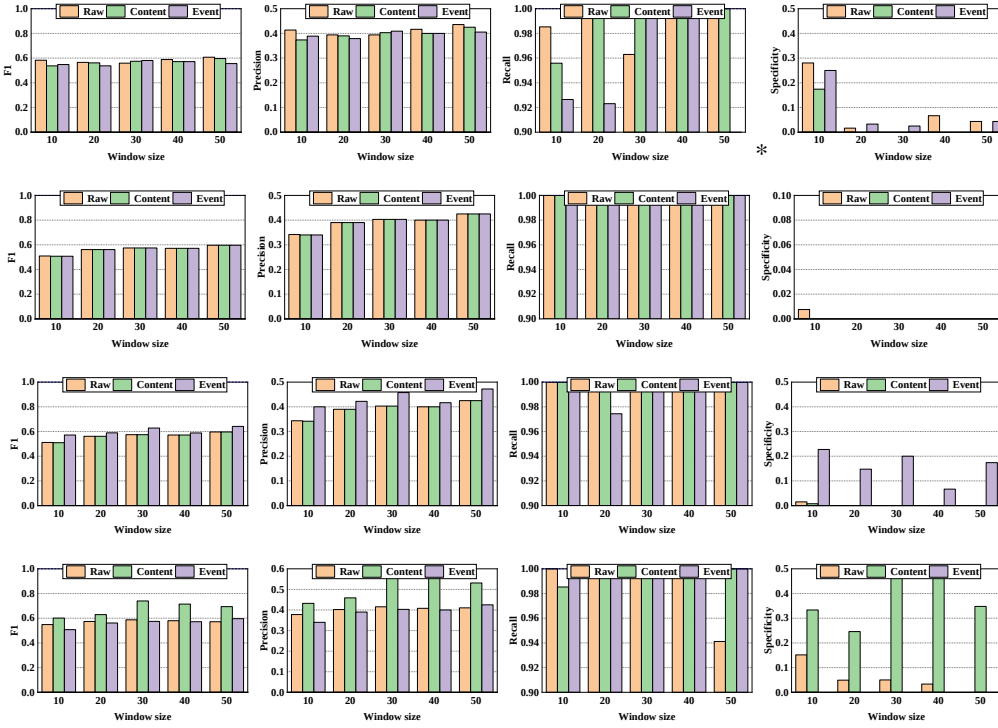


Fig. 8. Spirit. First row: Prompt-1 (zero-shot); Second row: Prompt-2 (zero-shot); Third row: Prompt-1 (few-shot); Last row: Prompt-2 (few-shot).